AI Finance Advisor – Defense Guide

Oct 12, 2026 · @Kaush

The 60-second pitch

The AI Finance Advisor is a web app that turns a user's real transactions into budgets, forecasts, warnings and plain-English advice. The core design idea is **"numbers from code, words from the LLM"**: every figure is computed in Python from the database, and the language model only explains those figures.

Say it like this:

> "Users link a (sandbox) bank, import a CSV, type transactions, or photograph a receipt. Everything lands in one PostgreSQL database. On top of that we run five AI/analytics features: a RAG chatbot grounded in the user's own data, an AI statement analyser, AI budget recommendations, a cash-flow forecast with a confidence band, and anomaly detection. The LLM is optional: with no API key, every feature still works through a deterministic reasoning layer, so the app never invents numbers and never goes down because an AI service is unreachable."

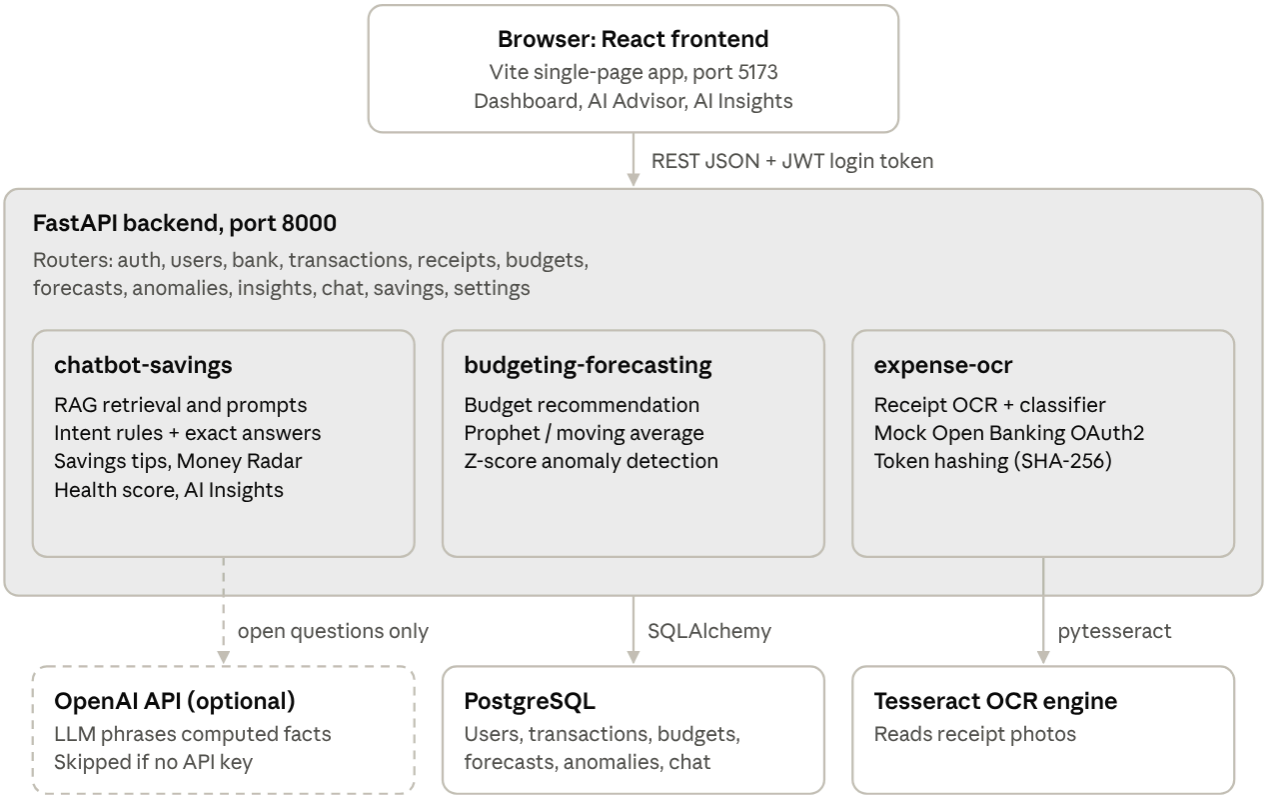
Where AI shows up in the app:

Feature	Page	Technique
AI Advisor chat	AI Advisor	RAG: retrieve the user's data, ground the LLM prompt, fall back to rule-based answers
AI Statement Insights	AI Insights	Statistics over a date range + an LLM-written summary
Budget recommendations	Dashboard	Spending history to per-category monthly limits
Cash-flow forecast	Dashboard	Prophet time-series model, moving-average fallback
Anomaly detection / Money Radar	Dashboard, AI Advisor	Z-score outliers + month-over-month spikes
Receipt scanning	Transactions	Tesseract OCR + text classification

Architecture

A three-tier web app: a React frontend, a FastAPI backend that holds all the logic, and a PostgreSQL database. The AI and analytics code lives in three subsystem folders, one per team member, that the backend imports.

Every request goes through FastAPI; the LLM is an optional last step



system architecture · 3 tiers, 3 subsystems

Each subsystem box is one team member's area. The dashed OpenAI link is the only external AI call, and it is optional.

How one request flows, e.g. the user asks "Where am I overspending?":

1. The React page sends `POST /chat/messages` with the user's JWT token.
2. FastAPI checks the token (`deps.get_current_user`) and loads that user's transactions and budgets from PostgreSQL via SQLAlchemy.
3. The chat router calls the chatbot-savings code, which computes the answer, or builds a RAG prompt for the LLM.
4. The answer, its sources and follow-up questions return as JSON, and the message is saved to `chat_messages` .

Tech stack: React 18 + Vite, FastAPI (Python 3.11), SQLAlchemy, PostgreSQL 16/17, bcrypt + JWT, Tesseract (pytesseract + Pillow), OpenAI SDK, ReportLab (PDF), Docker Compose (optional).

Who built what

Each member should be able to explain their own subsystem in depth and every other part at the level of this guide.

Member	Subsystem	AI / analytics features they own	Sections to master
Parth Patel (12281818)	Budgeting & Forecasting	Budget recommendation, cash-flow forecast (Prophet / moving average), z-score anomaly detection	Budgets, forecasting and anomaly detection
Preetkumar Navinbhai Patel (12277687)	Expense Tracking & OCR, Database & Backend Architecture	Receipt OCR + category classifier, mocked Open Banking OAuth2, database schema, FastAPI structure	Architecture; Receipt OCR and bank linking; Security
Thiwanka Kaushalya Nagasanga (12288198)	Conversational Chatbot (RAG) & Savings Recommendation	RAG chatbot, intent rules, savings suggestions, Money Radar, health score, AI Insights	AI chatbot (RAG); AI Insights; Savings suggestions

How the parts connect: OCR and bank sync **feed** transactions into PostgreSQL, Budgeting & Forecasting **turns** them into budgets, forecasts and anomaly flags, and the Chatbot **explains** all of it to the user. Use this one-line story when asked how the subsystems fit together.

AI chatbot (RAG)

The AI Advisor answers questions from the user's own transactions using Retrieval-Augmented Generation (RAG), with a deterministic fallback so it always gives a grounded answer. Code: `backend/api/routers/chat.py`, `backend/chatbot-savings/chatbot_prototype.py`, `backend/chatbot-savings/finance_insights.py`.

What RAG means here: instead of asking the LLM a bare question, we first *retrieve* the user's real data from PostgreSQL, *augment* the prompt with it, and only then let the model *generate*. The model is told to answer "using ONLY the transaction data provided", which stops it inventing numbers (hallucination).

What happens on `POST /chat/messages`:

1. **Load data.** The last 120 days of the user's transactions and their latest budget per category come from the database (`finance_data.py`). 120 days lets us compare this month with last month.
2. **Build a financial summary.** `compute_financial_summary()` works out this month's income, expenses, savings, savings rate, spend vs budget per category, and month-over-month change per category.
3. **Detect intent.** `detect_intent()` classifies the question with keyword rules into: affordability, what-if, savings goal, budget generator, overspending, savings tips, monthly summary, or general.
4. **Deterministic answer first.** For the seven known intents, Python computes the answer exactly from the summary, e.g. "Can I afford $200?" compares $200 with this month's savings. No LLM is involved, so these answers are always correct and instant.
5. **RAG + LLM for open questions.** If the intent is general:
   a. **Retrieve:** transactions in any category named in the question, or the 5 most recent if none is named.
   b. **Augment:** build a prompt with the financial summary, the retrieved transactions, the budget comparison and the savings suggestions.
   c. **Generate:** send it to the OpenAI API (`gpt-5`, up to 800 tokens).
6. **Graceful fallback.** With no `OPENAI_API_KEY`, or an empty reply, the user gets their real financial summary instead of an error.
7. **Transparency.** Every reply returns `sources` (e.g. "This month's transactions (18)", "Budget comparison across 4 categories"), shown as "Sources used" in the chat, plus 2–3 suggested follow-up questions based on the intent.
8. **Memory.** The question, answer and the retrieved context are saved to the `chat_messages` table.

Why this design (say this if asked):

- **Accuracy:** money figures come from Python, never from the model.
- **Reliability:** works offline and on campus networks that block the AI API.
- **Explainability:** the user can see what data grounded each answer.
- **Cost:** most common questions never call the paid API.

The prototype retrieval is keyword and recency based. Vector-similarity retrieval (embeddings) is the planned upgrade once there is enough transaction volume to need it.

AI Insights – statement analysis

AI Insights analyses any date range the user picks and returns cited findings, with a short AI-written summary on top. Code: `GET /insights/statement/analyze` and `POST /insights/statement/ask` in `backend/api/routers/insights.py`.

What the analyser computes:

Finding	How it is computed
Financial summary	Income, expenses, net cash flow, savings rate and transaction count for the range
Category breakdown	Spend per category, largest first
Unusual spending	Z-score test over a 180-day lookback; flags a transaction 2.5 or more standard deviations from that category's mean, then keeps only flags inside the range
Recurring charges	Groups by merchant; an average gap of 5–9, 12–16 or 25–35 days = weekly, fortnightly or monthly, with every gap within 40% of the average and every amount within 20%
Saving opportunities	Categories over their monthly budget, naming the merchant driving the overspend
Key insights	Short cards, each linked to the actual transactions ("View evidence")

The AI summary. The computed facts are turned into bullet lines and sent to the LLM with the instruction "Use ONLY the facts below - do not invent numbers". The response carries an `ai_generated` flag. The page labels it "AI-generated summary" or "Computed summary", so we never claim an LLM ran when it didn't.

Ask AI about this period. The same RAG pipeline as the chatbot, but retrieval is restricted to transactions inside the chosen range. The previous period is added only if the question asks for a comparison ("compare with last month"). The answer returns its evidence transactions.

Export. The same analysis downloads as CSV or PDF (ReportLab).

Budgets, forecasting and anomaly detection

These three features are statistical models over the user's history, built by the Budgeting & Forecasting subsystem (`backend/budgeting-forecasting/forecast_prototype.py`, `routers/budgets.py`, `routers/forecasts.py`, `routers/anomalies.py`).

AI budget recommendation (`POST /budgets/recommend`).

1. Take the last 3 complete calendar months of expenses, grouped by category and month (in SQL).
2. Compute a **recency-weighted average**: weights 1, 2, 3, so the most recent month counts most.
3. Apply a 5% savings target: recommended budget = weighted average × 0.95.
4. Save it as `generated_by='ai_engine'`, but never overwrite a budget the user already created, edited or deleted this month. The user stays in control.

Worked example: groceries of $300, $360 and $420 over three months gives (300×1 + 360×2 + 420×3) ÷ 6 = $380, then × 0.95 = **$361**.

Cash-flow forecast (`POST /forecasts`). Daily net cash flow (the sum of signed amounts per day) is turned into a running balance and projected 14 days ahead.

- **Prophet** (Meta's time-series model) is used when it is installed and there are at least 10 days of history. It models weekly seasonality and returns a confidence band (`yhat_lower` / `yhat_upper`).
- **Moving average** is the fallback: the average daily change over the last 14 days, added forward day by day. It gives no confidence band.
- The method used is stored (`model_version`) and shown on the Dashboard.
- Prophet is not in `requirements.txt`, so a default install runs the moving average. Say so if asked, and present Prophet as the upgrade path.

Anomaly detection (z-score). For each transaction:

```
z = \frac{|x - \mu_{category}|}{\sigma_{category}}
```

x is the amount, μ and σ are the mean and standard deviation of that category's transactions. A transaction is flagged when z ⩾ 3.0 on the Dashboard (180-day lookback), or z ⩾ 2.5 inside AI Insights. A category needs at least 3 transactions before anything is flagged, so one-off categories don't produce false alarms. Comparing within a category matters: $120 is normal for utilities but unusual for coffee.

Savings suggestions, Money Radar and health score

These turn the analysis into proactive, specific advice. The user doesn't have to ask a question. Code: `generate_savings_suggestions_detailed()` in `chatbot_prototype.py`, and `scan_money_radar()` and `financial_health_score()` in `finance_insights.py`.

Savings suggestions (`GET /savings/suggestions`). For each category over its budget this month, find the merchant driving the overspend and write a specific suggestion:

- **Several visits:** e.g. "You're $45.40 over your dining budget, largely driven by 3 visits to Corner Cafe ($38.10 total). Consider fewer visits a week or a cheaper alternative."
- **One large purchase:** asks whether it was a one-off or a pattern to budget for.

This is the key point to defend: the advice is **personalised and actionable**, not a generic "spend less". The same suggestions are also fed into the chatbot prompt, so the LLM has them as facts.

Money Radar (`GET /insights/money-radar`). It scans the user's finances each time the page loads, with no question needed. It raises three kinds of alert, sorted high severity first:

Alert	Rule	Severity
Spending more than you earn	Savings rate below 0% this month	High
Category over budget	Spent more than the budget	High if 20% or more over, else medium
Category spending rising	Up 50% or more vs last month, and at least $20 this month	Medium

Each alert carries a ready-made question (e.g. "How can I cut back on dining spending?"). The "Ask AI" button sends it straight into the chatbot. This links detection to explanation.

Financial health score (0–100). A transparent formula:

```
score = 50 + 0.75 \times clamp(savings\ rate, -40, 40) - 8 \times
(categories\ over\ budget)
```

The result is clamped to the 0–100 range. Example: a 20% savings rate with one category over budget gives 50 + 15 − 8 = **57**. It is deliberately simple and explainable, not a credit score.

Receipt OCR and bank linking

These are the two automated ways data gets in. OCR is computer vision plus text classification; bank linking is a mocked Open Banking OAuth2 flow. Code: `backend/expense-ocr/` and `routers/receipts.py`.

Receipt OCR (`POST /receipts/upload`).

1. **Save** the uploaded photo.
2. **Pre-process** it with Pillow: fix the phone's rotation (EXIF), convert to greyscale, upscale small images to at least 1000 px wide, and stretch the contrast.
3. **OCR** with Tesseract. If no total is found, run a second pass in single-block mode (`--psm 6`), which suits a column of receipt lines.
4. **Extract the total.** Take the last line labelled Total, Amount due or Balance due, and skip Subtotal, GST and Savings lines. Handles `$` , `1,234.56` and `45,20` . If no line is labelled, use the largest amount on the receipt.
5. **Extract the merchant:** the first line with at least 3 letters.
6. **Classify the category** by keyword matching, e.g. Woolworths or Coles = groceries, Uber or fuel = transport. Otherwise "uncategorised".
7. **Save** a `receipts` row (with the raw OCR text) and a negative `transactions` row with `source='receipt_ocr'` . It works with or without a linked bank.

The keyword classifier is a transparent baseline. A trained NLP classifier is the planned upgrade.

Bank linking (mocked Open Banking, OAuth2 authorisation-code flow).

1. The user ticks **consent** and signs in to the sandbox bank (`demo_user` / `demo1234`).
2. The backend issues an **authorisation code** plus a random `state` value. `state` protects against CSRF.
3. The callback exchanges the code for an **access token**. The code expires after 60 seconds.
4. Only a **SHA-256 hash** of the token is stored (`bank_accounts.external_ref`), never the raw token.
5. `POST /bank/sync` pulls the sandbox transactions in, skipping duplicates.

A real provider (e.g. a Consumer Data Right accredited API in Australia) would replace the mock without changing the rest of the app.

Security, privacy and responsible AI

The system treats financial data as sensitive and keeps the AI honest about what it knows.

Security and privacy:

- **Passwords** are hashed with bcrypt, never stored in plain text.
- **Sessions** use JWT bearer tokens that expire after 60 minutes. Every data endpoint requires one and filters by the logged-in `user_id`, so users can't see each other's data.
- **Bank tokens** are stored only as SHA-256 hashes.
- **Consent:** a `data_processing_consent` flag is required to link a bank and can be changed in Settings.
- **Right to erasure:** "Delete my data" removes the user, and `ON DELETE CASCADE` foreign keys remove all linked accounts, transactions, receipts, budgets, forecasts, anomalies and chat history. This aligns with the Australian Privacy Act and GDPR.
- **Secrets** (database password, JWT key, OpenAI key) live in a git-ignored `.env` file.

Responsible AI:

- **Grounding:** prompts say "use ONLY the data provided" and contain only the user's real figures.
- **No invented numbers:** all calculations are done in Python, and the LLM only phrases them.
- **Honest labelling:** the `ai_generated` flag shows "AI-generated" vs "Computed" in the UI.
- **Citations:** answers show their sources and evidence transactions.
- **Not financial advice:** prompts tell the model to frame suggestions as suggestions, and answers say so.
- **Minimal data to the LLM:** only the summary and the retrieved transactions are sent, not the full history or personal details. Without an API key, nothing leaves the server.

Likely examiner questions

Short answers you can say out loud. Each one points back to the code.

Q: What exactly is "AI" in this project, and isn't a lot of it just rules? There are three layers:

1. **Generative AI:** an LLM writes chat answers and the statement summary.

2. **Machine learning and statistics:** Prophet forecasting, z-score anomaly detection, weighted-average budget recommendation, and OCR, which is itself a neural-network model in Tesseract 4+.
3. **A deterministic reasoning layer** around them.

The rules are a deliberate design choice for accuracy and reliability, not a shortcut.

Q: What is RAG and why use it instead of just calling ChatGPT? Retrieval-Augmented Generation fetches the user's own data and puts it in the prompt. A plain LLM knows nothing about this user and would guess. RAG grounds the answer in real transactions, reduces hallucination, and lets us cite sources.

Q: How do you stop the LLM hallucinating numbers? Three ways:

- **Code does the maths:** the known question types never reach the LLM.
- **Strict prompt:** the instruction says "use ONLY the data provided".
- **Pre-computed facts:** the LLM gets finished figures and only phrases them.

Q: What happens if the OpenAI API is down or there's no key? Every AI feature has a fallback. Chat returns a deterministic answer or the real financial summary, and AI Insights shows a "Computed summary". The whole app can run fully offline.

Q: How does your retrieval work, and how would you improve it? Now: it matches category names in the question, otherwise takes the 5 most recent transactions, plus a full monthly summary. Next: embed transactions as vectors (e.g. pgvector in PostgreSQL) and retrieve by semantic similarity, so "coffee" finds "Corner Cafe".

Q: How is intent detected? Keyword rules in `detect_intent()`, e.g. "afford" gives affordability and "what if" gives what-if. They are fast, testable and explainable. An upgrade would be an LLM or a small trained classifier for intent.

Q: Why z-score for anomalies, and its weakness? It is simple, explainable ("3× your usual spend") and per category. It assumes roughly normal data and needs at least 3 transactions per category. Better: median/MAD or Isolation Forest.

Q: How are budgets recommended? A recency-weighted average of the last 3 months (weights 1:2:3), minus a 5% savings target. It never overwrites user edits.

Q: How do you know it works? Pytest integration tests against a real PostgreSQL database cover chat, savings suggestions, anomalies, bank sync, receipt upload, error handling and account deletion. There are also unit tests for the OCR classifier and the forecast prototype.

Q: How is user data protected when sent to the LLM? Only the needed summary and transactions are sent, with no name, email or account details. All requests are authenticated per user, and with no key nothing leaves the server.

Team questions (anyone may be asked):

Q: How did you integrate three subsystems built by different people? Each subsystem started as a standalone Python prototype with its own tests. The FastAPI backend imports them (`main.py` adds the three folders to the Python path) and wraps them as REST routes over one shared PostgreSQL schema (`db/schema.sql`). Chat, savings and insights share one transaction loader (`finance_data.py`), and every feature reads the same tables, so they all see identical data.

Q: Why FastAPI, React and PostgreSQL?

- **FastAPI:** Python, so the AI, statistics and OCR libraries plug in directly. It also gives automatic validation (Pydantic) and API docs at `/docs`.
- **React + Vite:** a fast, component-based single-page app.
- **PostgreSQL:** relational integrity for financial data, foreign keys with cascade delete, and date functions for monthly aggregation.

Q: Why not let the LLM do everything? LLMs are unreliable at arithmetic and can invent figures, which is unacceptable for money. They also cost money per call and need network access. So code computes and the LLM explains.

Q: What would you do with more time? Point to the Limitations and future work table: real Open Banking, vector retrieval, Prophet by default, trained classifiers, multi-turn memory and a cloud deployment.

Limitations and future work

State these openly before you're asked; examiners reward honesty about scope.

Limitation now	Future work
Bank linking is a mocked sandbox (`demo_user` / `demo1234`)	Connect a real Open Banking / Consumer Data Right provider
Retrieval is keyword and recency based	Vector embeddings + semantic search (e.g. pgvector)
Intent detection is keyword rules	A trained classifier or LLM function-calling
Prophet is optional, so a default install uses a moving average with no confidence band	Install Prophet by default and compare accuracy (MAE) against the baseline
Receipt category is keyword matching	A trained NLP / ML classifier on labelled receipts
Z-score assumes roughly normal spending	Median/MAD or Isolation Forest

Limitation now	Future work
Chat history is stored but not replayed into the next prompt	Multi-turn memory for follow-up questions
Runs locally (shared via a Cloudflare tunnel for demos)	Cloud deployment with HTTPS, a managed database and secret management